**SIMATS ENGINEERING**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

# ASSIGNMENT 3 – CO3

## C++ CONSTRUCTORS, DESTRUCTORS AND OPERATOR OVERLOADING

| | |
|---|---|
| **Student Name** | Keshavaram L |
| **Register Number** | 192511253 |
| **Department** | B.E. Computer Science and Engineering |
| **Assignment** | Assignment 3 |
| **Course Outcome** | CO3 |
| **Subject Name** | OOP WITH C++ |
| **Faculty Name** | MARY ANGES J |
| **GitHub Repository Link** | **https://github.com/keshavaram0104-boop/DSA0118_OOP-WITH-C-_192511253_KESHAVARAM-L** |

## ASSIGNMENT SCOPE AND EXECUTION NOTE

CO3 coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour, including default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

The uploaded question paper specifies individual execution, a GitHub repository named DSA01-<RegisterNumber>-Assignment3, one folder per task, a README.md, progressive commits, output screenshots, and a working repository link. The GitHub link is left blank on the cover page because no link was supplied.

Compiler used for program verification: g++ 14.2.0, C++17, with -Wall -Wextra -pedantic. The seven main programs below were compiled and executed. Their displayed outputs are the captured standard outputs from those runs.

Verification limitation: AddressSanitizer builds could be produced, but the execution environment reported that the ASan runtime did not come first in the initial library list; attempts to run the binaries then terminated before program output. Therefore, no sanitizer-clean claim or fabricated sanitizer report is made. Run the listed sanitizer commands on the student's own Linux/WSL environment and retain the actual report/screenshot.

### Recommended build command

```
g++ -std=c++17 -Wall -Wextra -pedantic Q1.cpp -o Q1
./Q1

# Memory-check build (run in a compatible local environment)
g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer Q2.cpp -o Q2_asan
./Q2_asan
```

### Contents


- Q1 – Constructors and Constructor Overloading
- Q2 – Copy Constructors: Shallow and Deep Copies
- Q3 – Dynamic Constructors and Destructors
- Q4 – Overloading Arithmetic and Stream Operators
- Q5 – Overloading Increment, Comparison and Subscript Operators
- Q6 – The Rules of Operator Overloading
- Q7 – Design a Complex-Number Class from First Principles

## Q1 – Constructors and Constructor Overloading (6 Marks)

### Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. The design must prevent an invalid number of copies.

### AIM

To design a Book class with default, parameterised and overloaded constructors, use member initializer lists and constructor delegation, and reject negative copy counts.

### EXPLANATION / THEORY

A constructor is a special member function called automatically when an object is created. A default constructor takes no arguments; a parameterised constructor accepts values; overloaded constructors provide several valid ways to initialise an object. The class declares title_, author_ and copies_ and routes simpler constructors to one validating constructor.

The compiler supplies an implicit default constructor only when the class has no user-declared constructor. Once any constructor is declared, the compiler does not automatically add a no-argument constructor. Thus, if a parameterised constructor is present, Book books[3] fails unless a default constructor is explicitly provided or every array element is otherwise initialised.

Member initializer lists are preferred because they directly initialise data members. Assignment in the constructor body first default-initialises members and then assigns to them; direct initialisation is required for const members and references and is often more efficient for class-type members.

Members are initialised in their declaration order, not the order written in the initializer list. Here title_ is initialised first, author_ second and copies_ third. The initializer list should follow that order to avoid confusion and -Wreorder warnings.

### ALGORITHM

- Declare title, author and copies as private members in a fixed order.
- Make the default, title-only and title-plus-copy constructors delegate to the full constructor.
- Initialise all members in the full constructor's member initializer list.

- Reject copies < 0 by throwing std::invalid_argument.
- Create one object using each constructor, print the states, and test an invalid count.

## C++ PROGRAM

```
#include <iostream>
#include <stdexcept>
#include <string>

class Book {
    std::string title_;
    std::string author_;
    int copies_;
public:
    Book() : Book("", "", 0) {}
    explicit Book(const std::string& title) : Book(title, "Unknown", 0) {}
    Book(const std::string& title, int copies) : Book(title, "Unknown", copies) {}
    Book(const std::string& title, const std::string& author, int copies)
        : title_(title), author_(author), copies_(copies) {
        if (copies_ < 0) throw std::invalid_argument("number of copies cannot be negative");
    }
    void print() const {
        std::cout << "Title: " << (title_.empty() ? "(empty)" : title_)
                  << ", Author: " << (author_.empty() ? "(empty)" : author_)
                  << ", Copies: " << copies_ << '\n';
    }
};

int main() {
    Book empty;
    Book donated("The C++ Primer");
    Book partial("Clean Code", 3);
    Book complete("Effective C++", "Scott Meyers", 5);
    empty.print();
    donated.print();
    partial.print();
    complete.print();
    try {
        Book invalid("Invalid Book", "Unknown", -2);
        invalid.print();
    } catch (const std::invalid_argument& e) {
        std::cout << "Rejected: " << e.what() << '\n';
    }
}
```

## OUTPUT

```
Title: (empty), Author: (empty), Copies: 0
Title: The C++ Primer, Author: Unknown, Copies: 0
Title: Clean Code, Author: Unknown, Copies: 3
Title: Effective C++, Author: Scott Meyers, Copies: 5
Rejected: number of copies cannot be negative
```

## State of the three requested object situations

| Object | Constructor used | State after construction |
|---|---|---|
| empty | Book() | title_ = ""; author_ = ""; copies_ = 0 |
| donated | Book("The C++ Primer") | title_ = "The C++ Primer"; author_ = "Unknown"; copies_ = 0 |
| complete | Book("Effective C++", "Scott Meyers", 5) | title_ = "Effective C++"; author_ = "Scott Meyers"; copies_ = 5 |

## EXPERIMENTAL INVESTIGATION

Removing the default constructor and compiling an array such as Book books[3] produces the compiler diagnostic: “error: no matching function for call to ‘Book::Book()’”. Each array element requires default initialisation, but the class has no zero-argument constructor.

A deliberately reversed initializer list, such as Reordered() : title("Example"), number(10) {} when number is declared before title, was compiled with -Wreorder. g++ reported: “Reordered::title will be initialized after” and identified number as the member that will be initialised first. The correct fix is to write the initializer list in declaration order.

The negative-count experiment is included in the program. The constructor throws std::invalid_argument before an invalid Book object can be used; the catch block prints the rejection.

## DESIGN QUESTION – explicit

Mark a single-argument constructor explicit when an unintended implicit conversion would be surprising or unsafe. For example, with Book(const std::string& title), the statement Book b = std::string("Networks"); is permitted if the constructor is non-explicit. If the constructor is declared explicit, copy-initialisation in that form is rejected; direct initialisation, Book b(std::string("Networks"));, remains valid.

## ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

A single constructor with defaults could be written as Book(std::string title = "", std::string author = "Unknown", int copies = 0). It can represent all three creation cases and still validate copies in one constructor body. Object construction takes O(1) time and O(1) additional space for either design (excluding the variable-length storage owned internally by std::string).

The overloaded-and-delegating design communicates the intended creation cases through distinct signatures and funnels validation into one full constructor. The default-argument design has one validation point as well, but can make calls with omitted middle arguments less expressive. Both designs can check copies in one place; delegation avoids duplicated validation.

## RESULT

The Book class was implemented and executed successfully. All valid construction forms produced the expected states, while a negative copy count was rejected.

## Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

### Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. A shallow copy duplicates only the pointer, causing two objects to refer to the same array; modifying one affects the other and both destructors may attempt to release the same allocation.

### AIM

To implement independent prescription copies using a deep-copying copy constructor, copy assignment operator and destructor, and to explain the Rule of Three.

### EXPLANATION / THEORY

A shallow copy copies the pointer value. Both objects then point to one allocation, so a change through either object is visible through the other. When both destructors call delete[] on that pointer, the program has a double-delete error and undefined behaviour.

A deep copy allocates a separate array and copies each medicine string. The original and copied objects own different arrays, so modifying one does not modify the other. The Rule of Three states that if a class needs a user-defined destructor, copy constructor or copy assignment operator because it manages a resource, it generally needs all three.

The copy constructor is used when a new object is initialised from an existing object, when an object is passed by value, and when an object is returned by value if copy elision does not remove the copy. Passing by const reference does not invoke the copy constructor.

### MEMORY REPRESENTATIONS

```
SHALLOW COPY (incorrect ownership)
Prescription p ----+----> [ medicine array ]
Prescription q ----+
Both pointers refer to the same allocation. Two destructors may delete it.

DEEP COPY (independent ownership)
Prescription p --------> [ Paracetamol | ORS ]
Prescription q --------> [ Paracetamol | ORS ]
The arrays occupy separate allocations; editing q does not change p.
```

### ALGORITHM

- Allocate an array of n strings in the constructor.
- In the copy constructor, allocate a new array of the same size and copy every element.
- In copy assignment, first check this != &other.
- Allocate and populate a fresh array before releasing the current array (stronger exception safety).

- Delete the old array, install the new pointer and size, and return *this.
- Use delete[] in the destructor.

## C++ PROGRAM – completed TODO 1 to TODO 3

```
#include <iostream>
#include <stdexcept>
#include <string>

class Prescription {
    int n_;
    std::string* medicines_;
    static int copies_;
public:
    explicit Prescription(int count)
        : n_(count), medicines_(count > 0 ? new std::string[count] : nullptr) {
        if (count < 0) throw std::invalid_argument("count cannot be negative");
    }
    Prescription(const Prescription& other)
        : n_(other.n_), medicines_(n_ > 0 ? new std::string[n_] : nullptr) {
        ++copies_;
        for (int i = 0; i < n_; ++i) medicines_[i] = other.medicines_[i];
    }
    Prescription& operator=(const Prescription& other) {
        if (this != &other) {
            std::string* fresh = other.n_ > 0 ? new std::string[other.n_] : nullptr;
            for (int i = 0; i < other.n_; ++i) fresh[i] = other.medicines_[i];
            delete[] medicines_;
            medicines_ = fresh;
            n_ = other.n_;
        }
        return *this;
    }
    ~Prescription() { delete[] medicines_; }
    void set(int i, const std::string& m) { medicines_[i] = m; }
    const std::string& get(int i) const { return medicines_[i]; }
    int size() const { return n_; }
    static void resetCopies() { copies_ = 0; }
    static int copyCount() { return copies_; }
};
int Prescription::copies_ = 0;

void byValue(Prescription p) {
    std::cout << "by value received " << p.size() << " medicines\n";
}
void byReference(const Prescription& p) {
    std::cout << "by const reference received " << p.size() << " medicines\n";
}

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | "
              << q.get(0) << ' ' << q.get(1) << " | "
              << r.get(0) << ' ' << r.get(1) << '\n';
    p = p;
    Prescription::resetCopies();
    byValue(p);
    byReference(p);
    std::cout << "Copy constructor calls in parameter test: "
              << Prescription::copyCount() << '\n';
```

```
    std::cout << "Self-assignment preserved: " << p.get(0) << '\n';
}
```

## OUTPUT

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
by value received 2 medicines
by const reference received 2 medicines
Copy constructor calls in parameter test: 1
Self-assignment preserved: Ibuprofen
```

The first line is the required completed-code result: Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS. It confirms that q and r retain their own copied contents after p is modified.

## EXPERIMENTAL INVESTIGATION

Copy-count experiment: passing Prescription by value made one copy in the observed call; passing by const reference made zero copies. The program resets and prints the copy-constructor counter around these calls. Self-assignment p = p is guarded and preserves the original medicine value.

| Medicine count | 100 deep copies (observed) | Complexity interpretation |
|---|---|---|
| 10 | 0.016335 ms | O(n) work per copy |
| 1,000 | 1.80018 ms | O(n) work per copy |
| 100,000 | 133.777 ms | O(n) work per copy |

These timings were measured with g++ -O2 in the preparation environment and are illustrative, not universal. Hardware, allocator behaviour and string lengths affect the result. Copying n medicine strings takes O(n) time and O(n) additional space.

Sanitizer verification status: the deep-copy source compiled with -fsanitize=address, but this execution environment could not start the ASan runtime (“ASan runtime does not come first in initial library list”) and the process terminated before the program ran. Therefore, no sanitizer pass is claimed. The shallow-copy version is expected to be diagnosed for double-free/use-after-free by a functioning ASan runtime; that report must be captured by running the deliberately shallow version locally.

## CONCLUSION

The compiler-generated copy constructor can be relied on when every member already has correct value semantics, such as integers, std::string and std::vector. A custom copy constructor is required when a class owns a raw resource whose default memberwise copy would duplicate ownership rather than the resource value.

## ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

If medicines were stored in std::vector<std::string>, the compiler-generated copy constructor and assignment operator would copy the vector's elements correctly. This is the Rule of Zero: let a resource-managing standard type manage its own memory and avoid writing the destructor, copy constructor and assignment operator.

| Design | Copy time | Extra space | Ownership |
|---|---|---|---|
| Raw pointer, deep copy | O(n) | O(n) | Correct when Rule of Three is implemented |
| std::vector<std::string> | O(n) | O(n) | Correct automatically |
| Shallow pointer copy | O(1) | O(1) | Incorrect shared ownership; double-delete risk |

A shallow pointer copy appears cheaper because it copies only an address, but it does not provide independent value semantics and is not a valid substitute for deep copying.

## RESULT

The completed program compiled and executed, and its output demonstrated independent copies, copy-by-value behaviour, const-reference behaviour and safe self-assignment. ASan runtime execution remains to be repeated in a compatible local environment.

## Q3 – Dynamic Constructors and Destructors (7 Marks)

### Problem Context – Auditorium Seat Booking

An auditorium seat map is created when an event is set up. The number of rows and seats per row varies. When the event object is destroyed, every allocation must be released and a closing summary printed.

### AIM

To allocate a two-dimensional seat map dynamically, book and cancel seats, and demonstrate destructor order for stack, block-scope and heap objects.

### EXPLANATION / THEORY

A dynamic constructor allocates memory at run time according to constructor arguments. In the row-pointer representation, bool** points to an array of row pointers; each row pointer points to a separately allocated array of bool values. The destructor releases each row with delete[] and then releases the row-pointer array with delete[].

A local object is destroyed automatically when its scope ends. An object inside a nested block is destroyed at the closing brace. A dynamically allocated object remains alive until delete is executed. Forgetting delete causes a memory leak. Using delete instead of delete[] for an array allocation is undefined behaviour.

A destructor has no return type and no parameters, so it cannot be overloaded. A class has one destructor, which may be defaulted or user-defined. Destruction occurs in reverse order of completed construction within a scope.

### MEMORY LAYOUT

```
Event object
  booked_ --> [row pointer 0] ----> [seat][seat][seat]
              [row pointer 1] ----> [seat][seat][seat]
              [row pointer 2] ----> [seat][seat][seat]

Allocation count for rows > 0: 1 row-pointer array + rows seat arrays.
Release order: delete[] each row, then delete[] booked_.
```

### ALGORITHM

- Validate that rows and seats are non-negative.
- If rows is positive, allocate an array of row pointers.
- Allocate and zero-initialise each row.
- Check row and seat bounds before booking or cancelling.
- In the destructor, delete every row and then the row-pointer array.

- Create stack, nested-block and heap events; explicitly delete the heap event.

## C++ PROGRAM

```
#include <iostream>
#include <stdexcept>
#include <string>

class Event {
    std::string name_;
    int rows_;
    int seats_;
    bool** booked_;
public:
    Event(const std::string& name, int rows, int seats)
        : name_(name), rows_(rows), seats_(seats), booked_(nullptr) {
        if (rows < 0 || seats < 0) throw std::invalid_argument("dimensions cannot be
negative");
        if (rows_ > 0) {
            booked_ = new bool*[rows_];
            for (int r = 0; r < rows_; ++r) booked_[r] = new bool[seats_]{};
        }
        std::cout << "Construct " << name_ << " (" << rows_ << "x" << seats_ << ")\n";
    }
    ~Event() {
        for (int r = 0; r < rows_; ++r) delete[] booked_[r];
        delete[] booked_;
        std::cout << "Destroy " << name_ << ": released " << rows_ * seats_ << " seats\n";
    }
    bool book(int r, int s) {
        if (r < 0 || r >= rows_ || s < 0 || s >= seats_ || booked_[r][s]) return false;
        booked_[r][s] = true;
        return true;
    }
    bool cancel(int r, int s) {
        if (r < 0 || r >= rows_ || s < 0 || s >= seats_ || !booked_[r][s]) return false;
        booked_[r][s] = false;
        return true;
    }
    int bookedCount() const {
        int count = 0;
        for (int r = 0; r < rows_; ++r)
            for (int s = 0; s < seats_; ++s) count += booked_[r][s] ? 1 : 0;
        return count;
    }
};

int main() {
    Event auditorium("Stack event", 2, 3);
    auditorium.book(0, 1);
    auditorium.book(1, 2);
    auditorium.cancel(0, 1);
    std::cout << "Stack event booked seats: " << auditorium.bookedCount() << '\n';
    {
        Event blockEvent("Block event", 1, 2);
        blockEvent.book(0, 0);
        std::cout << "Block event booked seats: " << blockEvent.bookedCount() << '\n';
    }
    Event* heapEvent = new Event("Heap event", 2, 2);
    heapEvent->book(1, 1);
    std::cout << "Heap event booked seats: " << heapEvent->bookedCount() << '\n';
    delete heapEvent;
    Event zero("Zero-row event", 0, 5);
    std::cout << "Zero-row event booked seats: " << zero.bookedCount() << '\n';
    std::cout << "End of main\n";
}
```

## OUTPUT

```
Construct Stack event (2x3)
Stack event booked seats: 1
Construct Block event (1x2)
Block event booked seats: 1
Destroy Block event: released 2 seats
Construct Heap event (2x2)
Heap event booked seats: 1
Destroy Heap event: released 4 seats
Construct Zero-row event (0x5)
Zero-row event booked seats: 0
End of main
Destroy Zero-row event: released 0 seats
Destroy Stack event: released 6 seats
```

## EXPERIMENTAL INVESTIGATION

The output records construction and destruction order. The block event is destroyed at the end of its block. The heap event is destroyed at the explicit delete statement. At the end of main, the zero-row local object is destroyed before the earlier-created stack event.

A zero-row event is handled without allocating the row-pointer array. The destructor's loop executes zero times and delete[] on the null pointer is safe.

| Seat map | Seat bytes | Row-pointer bytes (64-bit) | Approximate total |
|---|---|---|---|
| 10 × 20 | 200 | 80 | 280 bytes |
| 100 × 200 | 20,000 | 800 | 20,800 bytes |
| 1,000 × 2,000 | 2,000,000 | 8,000 | 2,008,000 bytes |

The estimates assume sizeof(bool) = 1 and sizeof(pointer) = 8 on the measured 64-bit platform. They exclude allocator metadata, alignment and object/string overhead. A row-pointer design performs rows + 1 allocations when rows is positive.

A missing delete on the heap-created Event leaks the object and its seat arrays. The ASan runtime could not execute in this preparation environment because its loader order was rejected; no sanitizer output is fabricated. The correct program was compiled and run normally.

## CONCLUSION – RAII

RAII (Resource Acquisition Is Initialization) ties resource lifetime to object lifetime. The constructor acquires the seat-map memory and the destructor releases it automatically when the object leaves scope. This reduces forgotten cleanup paths and makes stack and block-scope resources safer.

## ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

A single contiguous block stores rows × seats values and accesses element (r, s) at index r × seats + s. It requires one allocation and no row-pointer array. A std::vector<std::vector<char>> is also an option, with automatic cleanup but typically one vector allocation per row plus vector bookkeeping.

| Representation | Access | Allocation count | Additional row-pointer storage |
|---|---|---|---|
| bool** row arrays | O(1) | rows + 1 | rows × sizeof(bool*) |
| One contiguous block | O(1) | 1 | None |
| vector<vector<char>> | O(1) | Outer vector plus row storage | Vector metadata and row objects |

All forms provide O(1) indexed access. The contiguous block generally improves locality and reduces allocation overhead; the row-pointer layout allows rows to be allocated independently.

## PREDICT THE OUTPUT – Logger program

The exact output, captured by compiling and running the supplied Logger program, is:

```
+A +B -B copyAc use -Ac +P -P end -A
```

A is constructed first. B is constructed and destroyed at the nested block boundary. Passing a by value to use(Logger) creates a copy named Ac; the parameter is destroyed when use returns. P is dynamically created and destroyed by delete. The final local object A is destroyed when main ends.

## RESULT

The dynamic seat-map program compiled and executed. Its output confirms correct booking/cancellation, zero-row handling, explicit heap deletion and reverse-scope destruction order.

## Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

### Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts stored in rupees and paise. Operator overloading allows bill expressions to resemble ordinary arithmetic.

### AIM

To implement a Money class storing total paise in long long and overload arithmetic, comparison and stream operators.

### EXPLANATION / THEORY

The + operator returns a new Money value and leaves both operands unchanged. The += operator modifies the left operand and returns Money& referring to *this, which supports chaining and avoids an unnecessary copy. Equality and less-than compare the integer paise representation.

The insertion operator << is a non-member because the left operand is std::ostream, not Money. It is commonly declared friend so it can access the private paise value. The extraction operator >> is also a non-member stream operator. Member functions are suitable for +, -, +=, == and < in this design.

The expression a + b is translated conceptually to a.operator+(b) when the member overload is selected. A non-member alternative would be operator+(a, b).

### ALGORITHM

- Store every amount as one signed long long total-paise value.
- Implement + and - by combining total paise and constructing a new Money.
- Implement += by updating the current total and returning *this by reference.
- Compare stored paise for == and <.
- Format output as rupees and exactly two paise digits; preserve a negative sign.
- Read values through the stream extraction overload.

### C++ PROGRAM – completed TODO 1 and TODO 2

```
#include <iomanip>
#include <iostream>

class Money {
    long long paise_;
public:
    Money(long long rupees = 0, int paise = 0) : paise_(rupees * 100 + paise) {}
    explicit Money(long long totalPaise, bool) : paise_(totalPaise) {}
```

```
    Money operator+(const Money& rhs) const { return Money(paise_ + rhs.paise_, true); }
    Money operator-(const Money& rhs) const { return Money(paise_ - rhs.paise_, true); }
    Money& operator+=(const Money& rhs) { paise_ += rhs.paise_; return *this; }
    bool operator==(const Money& rhs) const { return paise_ == rhs.paise_; }
    bool operator<(const Money& rhs) const { return paise_ < rhs.paise_; }
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long value = m.paise_;
        if (value < 0) { os << "-Rs "; value = -value; }
        else os << "Rs ";
        return os << value / 100 << '.' << std::setw(2) << std::setfill('0')
                  << value % 100 << std::setfill(' ');
    }
    friend std::istream& operator>>(std::istream& is, Money& m) {
        long long rupees; int paise;
        if (is >> rupees >> paise) m.paise_ = rupees * 100 + paise;
        return is;
    }
};

int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
    Money carry = Money(0, 75) + Money(0, 50);
    std::cout << "Paise carry: " << carry << '\n';
    Money negative = Money(2, 0) - Money(3, 25);
    std::cout << "Negative subtraction: " << negative << '\n';
    Money bill(10, 50);
    bill += Money(2, 75);
    std::cout << "Updated bill: " << bill << '\n';
}
```

## OUTPUT

```
Rs 200.00
true
Paise carry: Rs 1.25
Negative subtraction: -Rs 1.25
Updated bill: Rs 13.25
```

## EXPERIMENTAL INVESTIGATION

The required carry test gives Rs 0.75 + Rs 0.50 = Rs 1.25. Subtracting Rs 3.25 from Rs 2.00 gives -Rs 1.25. This implementation permits signed balances; a billing application could instead reject a negative result as a business rule.

The overloaded bill expression is clearer: Money total = item1 + item2; total += tax;. Without overloading, named functions such as add(item1, item2) and addTo(total, tax) are more verbose.

| One-million additions | Observed elapsed time | Observed result |
|---|---|---|
| Integer paise | 0.609441 ms | 75,000,000 paise |
| double rupees | 4.81821 ms | 750,000.0 rupees |

This one-run -O2 microbenchmark is platform-dependent and is not a universal performance guarantee. Integer paise gives exact minor-unit arithmetic for values representable in long long. Binary floating point may not represent many decimal fractions exactly.

## ENGINEERING DECISION

Multiplying one Money amount by another Money amount is generally not meaningful: the result would have squared currency units. Multiplication by a scalar quantity or tax rate can be meaningful if rounding rules are specified. Addition, subtraction, equality and ordering are natural Money operations.

## ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

The arithmetic and comparison overloads could be non-member friend functions, or named functions such as add(a, b) and equals(a, b). Each operation is O(1) time and O(1) extra space. The main difference is interface flexibility and readability, not asymptotic complexity.

A non-member operator can support conversions on the left operand, such as 100 + m, as well as m + 100, when the overloads and conversion rules are deliberately designed for that purpose. A member operator is invoked on the left operand, so an ordinary integer left operand cannot call Money::operator+.

## RESULT

The Money program compiled and executed. The expected Rs 200.00 and true results were produced, and carry, negative subtraction and compound addition were verified.

## Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

### Problem Context – Music Playlist and Track Timer

A music player needs prefix and postfix increment for a Time object, comparisons between times, and range-checked playlist[i] access.

### AIM

To overload prefix/postfix ++, ==, < and both const and non-const operator[] overloads, then demonstrate playback and boundary checks.

### EXPLANATION / THEORY

The compiler distinguishes prefix and postfix increment by their signatures. Prefix is operator++() with no parameter. Postfix is operator++(int); the dummy int is a language convention and is not supplied explicitly by the programmer.

Prefix increments the object and returns *this by reference. Postfix first saves the old value, increments the object and returns the saved value by copy. Postfix therefore usually creates an extra object and can be slower for class types.

A non-const operator[] returns a modifiable reference, allowing playlist[i] = "Track". A const overload returns a const reference so a const Playlist can be read but not modified. An invalid index should not access memory; this implementation throws std::out_of_range for negative and too-large indices.

### ALGORITHM

- Store time as total seconds so minute and hour roll-over is automatic.
- Prefix ++ increments seconds and returns a reference to the updated object.
- Postfix ++ saves the old object, increments the current object and returns the saved value.
- Compare the stored seconds for == and <.
- Check the index in both Playlist subscript overloads before returning a reference.

### C++ PROGRAM

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

class Time {
    int seconds_;
public:
```

```
    explicit Time(int seconds = 0) : seconds_(seconds) {
        if (seconds < 0) throw std::invalid_argument("time cannot be negative");
    }
    Time& operator++() { ++seconds_; return *this; }
    Time operator++(int) { Time old(*this); ++seconds_; return old; }
    bool operator==(const Time& rhs) const { return seconds_ == rhs.seconds_; }
    bool operator<(const Time& rhs) const { return seconds_ < rhs.seconds_; }
    int totalSeconds() const { return seconds_; }
    friend std::ostream& operator<<(std::ostream& os, const Time& t) {
        int h = t.seconds_ / 3600, m = (t.seconds_ / 60) % 60, s = t.seconds_ % 60;
        return os << h << ':' << (m < 10 ? "0" : "") << m
                  << ':' << (s < 10 ? "0" : "") << s;
    }
};
class Playlist {
    std::vector<std::string> tracks_;
public:
    Playlist(std::initializer_list<std::string> tracks) : tracks_(tracks) {}
    std::string& operator[](int i) {
        if (i < 0 || i >= static_cast<int>(tracks_.size()))
            throw std::out_of_range("playlist index out of range");
        return tracks_[static_cast<std::size_t>(i)];
    }
    const std::string& operator[](int i) const {
        if (i < 0 || i >= static_cast<int>(tracks_.size()))
            throw std::out_of_range("playlist index out of range");
        return tracks_[static_cast<std::size_t>(i)];
    }
    int size() const { return static_cast<int>(tracks_.size()); }
};

int main() {
    Time t(3599);
    Time a = t++;
    Time b = ++t;
    std::cout << "a=" << a << ", b=" << b << ", t=" << t << '\n';
    std::cout << "Rollover: " << Time(3599) << " -> " << Time(3600) << '\n';
    Playlist playlist{"Opening", "Interlude", "Finale"};
    std::cout << "Playing: " << playlist[0] << ", " << playlist[1] << '\n';
    const Playlist& readOnly = playlist;
    std::cout << "Last track: " << readOnly[2] << '\n';
    try { std::cout << playlist[-1] << '\n'; }
    catch (const std::out_of_range& e) { std::cout << "Negative index: " << e.what() <<
'\n'; }
    try { std::cout << playlist[playlist.size()] << '\n'; }
    catch (const std::out_of_range& e) { std::cout << "Size index: " << e.what() << '\n'; }
    std::cout << std::boolalpha << "a < b: " << (a < b) << '\n';
}
```

## OUTPUT

```
a=0:59:59, b=1:00:01, t=1:00:01
Rollover: 0:59:59 -> 1:00:00
Playing: Opening, Interlude
Last track: Finale
Negative index: playlist index out of range
Size index: playlist index out of range
a < b: true
```

## PREDICT THE OUTPUT – Counter program

The supplied Counter program was compiled and executed. Its exact output is:

```
5 7 7
```

Starting from c = 5, c++ returns a copy containing 5 and then changes c to 6. ++c changes c to 7 and returns the updated object, so b and c both contain 7. Postfix is the version that makes the old-value copy.

## EXPERIMENTAL INVESTIGATION

The main program verifies a = t++ and b = ++t around the 59:59 boundary. The stored seconds are 3,599 initially; after postfix, a remains 3,599 and t becomes 3,600; after prefix, t and b become 3,601. Formatting converts 3,600 seconds to 1:00:00.

The program tests playlist[-1] and playlist[size]. Both throw std::out_of_range and are caught, so invalid indices do not access the vector.

| Build setting | 10,000,000 prefix increments | 10,000,000 postfix increments |
|---|---|---|
| -O0 | 32.1811 ms | 40.4008 ms |
| -O2 | 6.03694 ms | 5.18678 ms |

These are single-run measurements from a small benchmark using a volatile counter. Optimisation, compiler decisions, CPU scheduling and measurement noise can change the relationship; the semantic distinction remains that postfix returns the old value.

## CONCLUSION

Programmers often prefer ++i for class types because prefix returns a reference to the updated object and avoids constructing the old-value temporary. For built-in integer types, modern compilers generally optimise both forms effectively.

## ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

Playlist access could be provided by at(i), which throws std::out_of_range, or by an unchecked operator[] similar to a built-in array. Checked operator[] performs one additional bounds comparison and gives a clear error. at(i) has a familiar standard-library contract. An unchecked version is marginally simpler but makes invalid access undefined behaviour.

All access forms are O(1) per access. The range check adds O(1) work. For 10,000,000 accesses, the total remains O(n); the exact overhead should be measured with a benchmark that prevents the compiler from removing the accesses.

## RESULT

The Time and Playlist classes compiled and executed. Prefix/postfix values, comparison, both constness forms of subscript access, invalid indices and hour roll-over were demonstrated.

## Q6 – The Rules of Operator Overloading (7 Marks)

### Problem Context – A Simple Text Class for an SMS Gateway

The SMS gateway uses a Text class containing a dynamically allocated character array. It requires assignment, concatenation, comparison and printing with careful memory ownership.

### AIM

To implement Text with the Rule of Three and overload +, ==, != and << while preventing self-assignment errors.

### EXPLANATION / THEORY

The operators that cannot be overloaded are :: (scope resolution), . (member access), .* (pointer-to-member access), ?: (conditional operator) and sizeof. Their syntax or compile-time role is fundamental to the language.

Operator overloading cannot change an operator's precedence, associativity or arity. At least one operand must be a user-defined type; an overload cannot redefine an operation for two built-in types alone. Operator= must be a member function. An operator should be a friend when a non-member implementation needs access to private data, commonly for << where the left operand is a stream.

The Text class owns a raw character array, so it supplies a destructor, deep-copying copy constructor and copy assignment operator. Assignment checks self-assignment and builds the replacement buffer before deleting the old one.

### ALGORITHM

- Allocate strlen(text) + 1 characters and copy the null terminator.
- Deep-copy the character array in the copy constructor.
- In assignment, guard against this == &other and allocate the new buffer first.
- Concatenate by allocating enough space for both strings and the terminator.
- Compare with strcmp and print through a friend stream insertion operator.
- Release the owned array with delete[] in the destructor.

### C++ PROGRAM

```
#include <cstring>
#include <iostream>
#include <string>

class Text {
    char* data_;
```

```
    static int copies_;
public:
    explicit Text(const char* s = "") {
        const std::size_t n = std::strlen(s);
        data_ = new char[n + 1];
        std::memcpy(data_, s, n + 1);
    }
    Text(const Text& other) {
        const std::size_t n = std::strlen(other.data_);
        data_ = new char[n + 1];
        std::memcpy(data_, other.data_, n + 1);
        ++copies_;
    }
    Text& operator=(const Text& other) {
        if (this != &other) {
            const std::size_t n = std::strlen(other.data_);
            char* fresh = new char[n + 1];
            std::memcpy(fresh, other.data_, n + 1);
            delete[] data_;
            data_ = fresh;
        }
        return *this;
    }
    ~Text() { delete[] data_; }
    Text operator+(const Text& rhs) const {
        const std::size_t a = std::strlen(data_), b = std::strlen(rhs.data_);
        char* joined = new char[a + b + 1];
        std::memcpy(joined, data_, a);
        std::memcpy(joined + a, rhs.data_, b + 1);
        Text result(joined);
        delete[] joined;
        return result;
    }
    bool operator==(const Text& rhs) const { return std::strcmp(data_, rhs.data_) == 0; }
    bool operator!=(const Text& rhs) const { return !(*this == rhs); }
    friend std::ostream& operator<<(std::ostream& os, const Text& t) { return os << t.data_;
}
    static int copyCount() { return copies_; }
    static void resetCopies() { copies_ = 0; }
};
int Text::copies_ = 0;

int main() {
    Text hello("Hello "), world("World"), empty("");
    Text message = hello + world;
    Text longText("This is a longer message used to verify dynamic allocation safely.");
    std::cout << "Message: " << message << '\n';
    std::cout << "Empty message: [" << empty << "]\n";
    std::cout << "Long message: " << longText << '\n';
    std::cout << std::boolalpha << "message != hello: " << (message != hello) << '\n';
    message = message;
    std::cout << "Self-assignment: " << message << '\n';
    Text::resetCopies();
    Text chain = hello + world + Text("!");
    std::cout << "Chain: " << chain << '\n';
    std::cout << "Copy constructor calls in chain: " << Text::copyCount() << '\n';
    const std::string stdA = "Hello ", stdB = "World";
    std::cout << "std::string equivalent: " << stdA + stdB << '\n';
}
```

## OUTPUT

```
Message: Hello World
Empty message: []
Long message: This is a longer message used to verify dynamic allocation safely.
message != hello: true
Self-assignment: Hello World
Chain: Hello World!
```

```
Copy constructor calls in chain: 0
std::string equivalent: Hello World
```

## EXPERIMENTAL INVESTIGATION

The program tests an empty message, a longer message, comparison, self-assignment and a + b + c. In C++17, guaranteed copy elision for returned prvalues means the observed copy-constructor counter for this expression is zero. This does not mean no characters are copied: the concatenation still allocates and copies character data.

| 100,000 concatenations | Observed time (-O2) |
|---|---|
| Custom Text | 9.10617 ms |
| std::string | 2.69027 ms |

The benchmark repeatedly concatenated short strings. The standard library implementation was faster in this run; the result is platform-dependent. The Text implementation is educational and omits optimisations such as capacity management and move operations.

The custom Text source compiled and ran normally. The ASan binary could not execute in this environment because of the runtime-loader issue described earlier; no sanitizer-clean claim is made. Run the same build with -fsanitize=address locally.

## CONCLUSION

Writing Text demonstrates ownership, deep copying, self-assignment protection, exception-safe replacement and the cost of manual memory management. In production, std::string should normally be preferred. A custom string-like class is justified only when specialised storage, representation, interoperability or resource constraints require it.

## ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

Replacing Text with std::string applies the Rule of Zero and supplies mature memory management, copying and concatenation. Another implementation is Text operator+(Text a, const Text& b) { a += b; return a; }. This passes the left operand by value, appends to that copy and returns it; an += implementation must manage capacity and allocation.

For a + b + c, let the operand lengths be n1, n2 and n3. The first concatenation copies approximately n1 + n2 characters. The second copies approximately (n1 + n2) + n3 characters. Total character-copy work is O(n1 + n2 + n3), with a temporary holding the first combined string and another result for the full expression. A design using repeated immutable + can allocate intermediate buffers; a reserved mutable builder or append operation can reduce allocations.

| Approach | Character work | Temporary storage |
|---|---|---|
| Text operator+ returning a new value | O(n1+n2+n3) | Intermediate combined string plus final result |
| Pass-left-by-value + += implementation | O(n1+n2+n3) amortised only if capacity strategy is good | One mutable left-hand copy, possible reallocation |
| std::string | O(n1+n2+n3) for this expression | Managed temporary results; move/elision optimisations |

## RESULT

The Text class compiled and executed, handled empty and long strings, protected self-assignment, and demonstrated the effect of C++17 copy elision.

## Q7 – Design a Complex-Number Class from First Principles (10 Marks)

### Problem Context – AC Circuit Impedance Calculator

AC circuit impedances are complex values. A resistor has ZR = R + 0j, an inductor has ZL = 0 + jωL, and a capacitor has ZC = 0 − j/(ωC). Series impedances add; parallel impedances combine using (Z1 × Z2) / (Z1 + Z2). The calculator must print rectangular form and magnitude and handle division by zero safely.

### AIM

To design and implement a Complex class from first principles and use it to calculate series and parallel RLC impedances.

### STAGE 1 – INTERPRET THE PROBLEM

Data required: real part, imaginary part, frequency f, angular frequency ω, resistance R, inductance L and capacitance C. The class needs a default constructor, a constructor from a real part, and a constructor from real and imaginary parts.

Required operations are addition, subtraction, multiplication, division, equality, rectangular stream output and magnitude. Division must detect a zero denominator.

| Requirement | Measurable acceptance statement |
|---|---|
| Construction | Default gives 0 + j0; one-argument gives x + j0; two-argument preserves both parts. |
| Arithmetic | +, -, *, / match the complex-number formulas. |
| Equality | Comparison uses a stated floating-point tolerance. |
| Display | Output shows real and signed imaginary parts in rectangular form. |
| Magnitude | Magnitude equals sqrt(real² + imaginary²). |
| Safety | Division by 0 + j0 throws a domain error rather than dividing by zero. |

### STAGE 2 – DESIGN

The real and imaginary components are private double members. Arithmetic operators are members because they operate on a left-hand Complex object. The stream insertion operator is a friend non-member because its left operand is std::ostream. The class also provides magnitude() as a named operation.

| Operator | Formula |
|---|---|
| + | (a + jb) + (c + jd) = (a+c) + j(b+d) |
| - | (a + jb) − (c + jd) = (a−c) + j(b−d) |
| * | (a + jb)(c + jd) = (ac−bd) + j(ad+bc) |
| / | ((ac+bd) + j(bc−ad)) / (c²+d²), where c²+d² ≠ 0 |
| == | Both components differ by less than the chosen tolerance |

| Magnitude | $\|z\| = \sqrt{a^2+b^2}$ |
|---|---|

## STAGE 3 – IMPLEMENT AND VERIFY

### ALGORITHM

- Calculate angular frequency $\omega = 2\pi f$.
- Construct ZR = (R, 0), ZL = (0, ωL), and ZC = (0, −1/(ωC)).
- Calculate series impedance Zseries = ZR + ZL + ZC.
- Calculate the three-branch parallel impedance as (ZR × ZL × ZC) / (ZR×ZL + ZR×ZC + ZL×ZC).
- Print each impedance and its magnitude.
- Attempt division by zero and catch std::domain_error.

### C++ PROGRAM

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>

class Complex {
    double re_, im_;
public:
    Complex() : re_(0.0), im_(0.0) {}
    Complex(double real) : re_(real), im_(0.0) {}
    Complex(double real, double imag) : re_(real), im_(imag) {}
    double real() const { return re_; }
    double imag() const { return im_; }
    double magnitude() const { return std::hypot(re_, im_); }
    Complex operator+(const Complex& z) const { return {re_ + z.re_, im_ + z.im_}; }
    Complex operator-(const Complex& z) const { return {re_ - z.re_, im_ - z.im_}; }
    Complex operator*(const Complex& z) const {
        return {re_ * z.re_ - im_ * z.im_, re_ * z.im_ + im_ * z.re_};
    }
    Complex operator/(const Complex& z) const {
        const double denominator = z.re_ * z.re_ + z.im_ * z.im_;
        if (denominator == 0.0) throw std::domain_error("division by zero impedance");
        return {(re_ * z.re_ + im_ * z.im_) / denominator,
                (im_ * z.re_ - re_ * z.im_) / denominator};
    }
    bool operator==(const Complex& z) const {
        const double eps = 1e-9;
        return std::abs(re_ - z.re_) < eps && std::abs(im_ - z.im_) < eps;
    }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
        os << std::fixed << std::setprecision(4) << z.re_;
        if (z.im_ < 0) os << " - j" << std::abs(z.im_);
        else os << " + j" << z.im_;
        return os;
    }
};

int main() {
    const double pi = std::acos(-1.0), f = 50.0, w = 2.0 * pi * f;
    const double R = 10.0, L = 0.1, C = 100e-6;
    const Complex zr(R), zl(0.0, w * L), zc(0.0, -1.0 / (w * C));
    const Complex series = zr + zl + zc;
    const Complex parallel = (zr * zl * zc) / (zr * zl + zr * zc + zl * zc);
    std::cout << "Series RLC impedance: " << series << " ohm\n";
```

```
    std::cout << "Series magnitude: " << std::fixed << std::setprecision(4)
              << series.magnitude() << " ohm\n";
    std::cout << "Parallel RLC impedance: " << parallel << " ohm\n";
    std::cout << "Parallel magnitude: " << parallel.magnitude() << " ohm\n";
    try {
        const Complex invalid = zr / Complex();
        std::cout << invalid << '\n';
    } catch (const std::domain_error& e) {
        std::cout << "Safe division: " << e.what() << '\n';
    }
}
```

## OUTPUT

```
Series RLC impedance: 10.0000 - j0.4151 ohm
Series magnitude: 10.0086 ohm
Parallel RLC impedance: 9.9998 + j0.0415 ohm
Parallel magnitude: 9.9999 ohm
Safe division: division by zero impedance
```

## HAND VERIFICATION

Given f = 50 Hz, R = 10 Ω, L = 0.1 H and C = 100 μF:

```
ω = 2πf = 314.1593 rad/s
XL = ωL = 31.4159 Ω
XC = 1/(ωC) = 31.8310 Ω
Zseries = 10 + j(XL − XC) = 10 − j0.4151 Ω
|Zseries| = sqrt(10² + 0.4151²) ≈ 10.0086 Ω
```

For the parallel circuit, the program evaluates the equivalent impedance of the resistor, inductor and capacitor branches using the three-branch product-over-pairwise-sum identity. The executed result is approximately 9.9998 + j0.0415 Ω, with magnitude 9.9999 Ω. Small differences in the last printed digit can arise from floating-point rounding.

The zero-impedance test throws std::domain_error and prints “division by zero impedance”, so the program does not silently divide by zero.

## STAGE 4 – DEFEND THE DESIGN

std::complex<double> is a standard, tested implementation and is preferable for ordinary engineering software. The custom class makes the representation and formulas explicit and supports the learning objective, but it requires careful testing and maintenance. The default constructor represents zero; the real-only constructor represents a purely real impedance; the two-component constructor represents any rectangular complex value. The arithmetic operators express the circuit formulas directly.

One remaining limitation is floating-point precision: double values are approximate, and exact equality is not suitable for most computed engineering values. This implementation uses a tolerance of 1e-9 for equality. It also checks an exactly zero denominator; a production numerical library may use scaled algorithms for improved stability.

## ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

Using std::complex<double> replaces the custom class with the standard type and includes <complex>. The series expression becomes std::complex<double> series = zr + zl + zc;. The parallel expression can use (zr * zl * zc) / (zr * zl + zr * zc + zl * zc).

| Measure | Custom Complex | std::complex<double> |
|---|---|---|
| Time per arithmetic operation | O(1) | O(1) |
| Storage | Two doubles | Two doubles in the usual implementation |
| Accuracy | double precision; custom tolerance | double precision; standard implementation |
| Division by zero | Explicit domain_error check | Caller must check denominator/handle result according to application needs |
| Implementation work | Must design, implement and test | Provided by the standard library |

## RESULT

The Complex class compiled and executed. Series and parallel impedances, magnitudes, hand calculations and safe division-by-zero handling were demonstrated.

## FINAL VERIFICATION AND SUBMISSION CHECKLIST

All seven C++17 source programs were compiled with g++ 14.2.0 using -Wall -Wextra -pedantic. No compiler warnings were emitted for the seven main programs. Each program was executed and the OUTPUT sections reproduce the captured standard output.

| Question | Program compiled | Program executed | Key verified points |
|---|---|---|---|
| Q1 | Yes | Yes | All constructors, delegation, negative count rejection |
| Q2 | Yes | Yes | Deep copy, assignment, self-assignment, by-value copy count |
| Q3 | Yes | Yes | Dynamic rows, booking/cancel, destructor order, zero rows |
| Q4 | Yes | Yes | Arithmetic, comparison, formatting, paise carry, negative result |
| Q5 | Yes | Yes | Prefix/postfix, comparisons, checked const/non-const subscript, rollover |
| Q6 | Yes | Yes | Rule of Three, concatenation, comparison, self-assignment, empty text |
| Q7 | Yes | Yes | Complex arithmetic, RLC calculations, magnitude, zero division |

AddressSanitizer note: the ASan binaries built, but the sandbox loader/runtime prevented execution. This is an environment limitation, not evidence that the programs passed ASan. Before final college submission, run the following on a compatible system and attach the real terminal output or screenshots:

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer Q2.cpp -o Q2_asan &&
./Q2_asan
g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer Q3.cpp -o Q3_asan &&
./Q3_asan
g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer Q6.cpp -o Q6_asan &&
./Q6_asan
```

### GitHub repository submission items from the question paper

- Create DSA01-192511253-Assignment3 and share it with the course faculty.
- Create one folder per question (Q1 through Q7) and place the corresponding .cpp file in it.
- Add input files and genuine output screenshots where applicable.
- Add README.md with student name, register number, compiler/version, exact compile commands, run steps and a short result for each task.
- Commit progressively, with at least one commit per task.
- Enter the working GitHub repository URL on the cover page and in the submission field.

The assignment report is complete as a written answer document. The GitHub repository, faculty sharing, progressive commits and local sanitizer screenshots are external submission actions and have not been represented as completed.